Comprehensive Formalization and Algorithmic Analysis of Subarray Problems: Prefix Transformations, Sliding Windows, Dynamic Programming, Monotonic Structures, and Bitwise Monotonicity

Abstract

This paper presents a formal, graduate-level algorithmic analysis and taxonomy of contiguous subarray problems across computational algorithms and competitive programming (LeetCode). Subarray problems form a core computational primitive where contiguous sequence bounds require optimal space-time trade-offs. We systematically categorize subarray problems into six foundational paradigms: (1) Prefix-Sum and Hash-Map Equivalence Classes (O(N) exact sum, remainder, modulo, and parity matching), (2) Two-Pointer Sliding Window Contractions (O(N) greedy optimization under monotonic domain constraints), (3) Dynamic Programming & Kadane's Local Optimization (O(N) maximum/minimum/product subarray state transitions), (4) Monotonic Stack & Element Contribution Mechanics (O(N) range boundaries and subarray count/sum optimizations), (5) Divide-and-Conquer & Range Trees (O(N log N) range sum counting with negative values), and (6) Bitwise Monotonicity (O(30N) bounded transitions for OR/AND/XOR). For each paradigm, we present mathematical invariant proofs, complexity theorems, edge-case taxons, detailed execution traces, and compilable Java reference implementations. Finally, we synthesize a diagnostic decision tree and master taxonomy matrix enabling immediate classification and optimal solution derivation for all contiguous subarray problems.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction and Mathematical Preliminaries

Contiguous subarray problems represent a foundational computational primitive in algorithm design, data structures, dynamic programming, and competitive programming platforms such as LeetCode. A central challenge in sequence processing is evaluating metrics—such as sum, product, parity, distinct element counts, minimums, maximums, or bitwise combinations—over bounded contiguous segments of an array.

While brute-force enumeration of all candidate subarrays requires quadratic $\mathcal{O}(N^2)$ candidate evaluations and $\mathcal{O}(N^3)$ naive computation time, optimal algorithms achieve linear $\mathcal{O}(N)$ or linearithmic $\mathcal{O}(N \log N)$ execution time by exploiting structural invariants, algebraic equivalences, monotonic state transitions, and memory-efficient data structures.

This research paper presents a formal, exhaustive, graduate-level taxonomy and algorithmic analysis of contiguous subarray problems. We establish six fundamental paradigms that completely govern the design of optimal subarray algorithms:

1) Prefix-Sum Space and Hash-Map Equivalence Classes (§2): Decomposing contiguous subarray metrics into prefix state differences, enabling exact sum matching, divisibility classes, zero-sum optimizations, and domain transformations in $\mathcal{O}(N)$ time.
2) Sliding Window and Two-Pointer Search Space Pruning (§3): Exploiting monotonic contraction properties across non-negative domains to prune candidate states from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, including exact-K inclusion-exclusion decompositions and monotonic deque window bounds.
3) Dynamic Programming and Kadane's Local State Optimization (§4): Formalizing optimal sub-structure for maximum/minimum subarray sums, product propagation under multiplicative sign inversion, circular topology transformations, and single-element deletion DP models.
4) Monotonic Stack and Element Contribution Mechanics (§5): Inverting search focus from subarray iteration to element contribution, deriving exact left/right boundary ranges (PSE and NSE) to evaluate sum of subarray minimums, maximums, and range bounds in $\mathcal{O}(N)$ time.
5) Divide-and-Conquer, Range Trees, and Monotonic Deques on Negative Prefixes (§6): Resolving non-monotonic prefix spaces for range sum bounds ($\mathcal{O}(N \log N)$ merge-sort divide and conquer) and shortest target sums with negative numbers ($\mathcal{O}(N)$ monotonic deque on prefix sums).

6) Bitwise Monotonicity and Bounded State Transitions (§7): Exploiting the 32-bit accumulation boundary of bitwise OR and AND operations to constrain distinct subarray bitwise states to $\mathcal{O}(30N)$.

Furthermore, we provide complete, compilable Java reference implementations for every canonical problem class, formal mathematical proofs of invariant correctness, and synthesize a diagnostic decision tree and taxonomy matrix (§8) for immediate classification of any LeetCode subarray problem.

1.1 Formal Definitions and Notation

Let $A = [a_0, a_1, \ldots, a_{N-1}]$ be a sequence of N real or integer elements indexed from 0 to $N-1$.

Definition 1 (Contiguous Subarray). A contiguous subarray $A[i \ldots j]$ defined by start boundary i and end boundary j (where $0 \le i \le j < N$) is the ordered subsequence of elements:

$$A[i \ldots j] = (a_i, a_{i+1}, \ldots, a_j) \tag{1}$$

Theorem 2 (Cardinality of Subarray Space). The total number of non-empty contiguous subarrays in an array of size N is exactly $\frac{N(N+1)}{2} = \mathcal{O}(N^2)$.

Proof. A contiguous subarray is uniquely identified by choosing a pair of indices (i, j) such that $0 \le i \le j < N$. Summing the number of valid ending indices j for each starting index i:

$$\sum_{i=0}^{N-1} (N - i) = N + (N-1) + \cdots + 1 = \frac{N(N+1)}{2} = \frac{N^2 + N}{2} = \mathcal{O}(N^2) \tag{2}$$

□

Definition 3 (Prefix Sum Array). The prefix sum array P of sequence A is defined for $r \in [0, N-1]$ as:

$$P[r] = \sum_{k=0}^{r} a_k \quad \text{with base definition } P[-1] = 0 \tag{3}$$

Lemma 4 (Subarray Sum Decomposition). The sum of elements in contiguous subarray $A[i \ldots j]$, denoted $S(i, j)$, equals the difference of prefix sums:

$$S(i, j) = \sum_{k=i}^{j} a_k = P[j] - P[i-1] \quad \text{for } 0 \le i \le j < N \tag{4}$$

Proof. Expanding prefix sums:

$$P[j] - P[i-1] = \sum_{k=0}^{j} a_k - \sum_{k=0}^{i-1} a_k = \sum_{k=i}^{j} a_k = S(i, j) \tag{5}$$

□

2 Paradigm 1: Prefix-Sum Space and Hash-Map Equivalence Classes

When an algorithm requires identifying subarrays whose sum matches a target K, equals zero, or satisfies a modulo divisibility condition $S(i, j) \pmod K = 0$, direct iteration yields $\mathcal{O}(N^2)$ time. By applying Lemma 1, target subarray conditions are mapped to prefix state algebraic equations solved via hash maps in $\mathcal{O}(N)$ time.

2.1 Target Sum Matching ($P[j] - P[i-1] = K$)

To find the count of subarrays with sum K, we rewrite $S(i, j) = K$ as:

$$P[j] - P[i-1] = K \iff P[i-1] = P[j] - K \tag{6}$$

As we iterate j from 0 to $N-1$, $P[j]$ is computed in $\mathcal{O}(1)$. The number of valid left boundaries $i-1$ ending prior to j equals the frequency of the value $(P[j] - K)$ observed in the prefix sequence up to $j-1$.

2.1.1 Subarray Sum Equals K (LeetCode 560)

We maintain a hash map storing the frequency of encountered prefix sums, initialized with $\{0 \to 1\}$ to account for $P[-1] = 0$.

Theorem 5 (Completeness of Prefix Hash Map Search). A single-pass iteration maintaining prefix sum frequencies correctly counts all valid subarrays summing to K without overcounting or undercounting.

Proof. For any right index j, every valid subarray $A[i \ldots j]$ with sum K corresponds to a unique prior prefix index $i - 1 < j$ where $P[i-1] = P[j] - K$. Because the hash map stores the exact count of indices $k < j$ where $P[k] = P[j] - K$, adding this count at step j accounts for all valid subarrays ending at j. Summing over all $j \in [0, N-1]$ yields the total count of valid subarrays. □

2.1.2 Longest Subarray Zero Sum (Screenshot Problem / Scaler / LeetCode)

When the target sum is $K = 0$, the requirement simplifies to $P[i-1] = P[j]$. To maximize the subarray length $(j - i + 1) = j - (i - 1)$, we must find the earliest index $i - 1$ where $P[i-1] = P[j]$.

Theorem 6 (Optimal Length Maximization Invariant). To maximize subarray length for a target prefix sum match, the hash map must store strictly the first occurrence index of each prefix sum value and never overwrite existing entries.

Proof. Let $idx_1 < idx_2$ be two indices where $P[idx_1] = P[idx_2] = V$. For a future index $j > idx_2$ where $P[j] = V$, the candidate subarray lengths are $L_1 = j - idx_1$ and $L_2 = j - idx_2$. Since $idx_1 < idx_2$, $L_1 > L_2$. Retaining idx_1 guarantees maximum length. □

2.2 Modulo and Remainder Equivalence Classes $(S(i,j) \pmod K = 0)$

For divisibility problems, $S(i,j) \pmod K = 0$ translates via prefix sums to:

$$(P[j] - P[i-1]) \pmod K = 0 \iff P[j] \pmod K = P[i-1] \pmod K \tag{7}$$

2.2.1 Canonical Negative Modulo Normalization

In programming languages such as Java or C++, the modulo operator ‘

$$R(P) = ((P \pmod K) + K) \pmod K \tag{8}$$

2.2.2 Continuous Subarray Sum (LeetCode 523)

Given length constraint ≥ 2, we store the first occurrence index of remainder R. If R was previously seen at index $firstIdx$ and $j - firstIdx \geq 2$, a valid continuous subarray of length ≥ 2 exists.

2.3 Domain Transformation: Equal 0s and 1s & Parity Matching

2.3.1 Contiguous Array (LeetCode 525)

To find the longest subarray with equal numbers of 0s and 1s, transform sequence A by mapping $0 \to -1$ and $1 \to 1$. An equal count of 0s and 1s becomes a sum of 0. The problem reduces directly to the Longest Subarray Zero Sum algorithm.

2.3.2 Subarrays with Odd Sum (LeetCode 1524)

Parity arithmetic dictates:

$$\text{Odd} - \text{Even} = \text{Odd} \quad \text{and} \quad \text{Even} - \text{Odd} = \text{Odd} \tag{9}$$

If current prefix sum $P[j]$ is odd, any prior even prefix sum forms an odd subarray. If $P[j]$ is even, any prior odd prefix sum forms an odd subarray. We maintain scalar counts odd_cnt and $even_cnt$, achieving $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.

2.4 Compilable Java Reference Implementations

```
import java.util.HashMap;
import java.util.Map;

public class PrefixSubarraySolutions {

    // 1. Longest Subarray with Zero Sum
    public static int longestSubarrayZeroSum(int[] A) {
        Map<Long, Integer> firstSeen = new HashMap<>();
        firstSeen.put(0L, -1); // Base case for prefix sum 0 at index -1
        long pref = 0;
        int maxLen = 0;

        for (int j = 0; j < A.length; j++) {
            pref += A[j];
            if (firstSeen.containsKey(pref)) {
                maxLen = Math.max(maxLen, j - firstSeen.get(pref));
            } else {
                firstSeen.put(pref, j);
            }
        }
        return maxLen;
    }

    // 2. Subarray Sum Equals K (LeetCode 560)
```

```
    public static int subarraySumEqualsK(int[] A, int K) {
        Map<Long, Integer> freq = new HashMap<>();
        freq.put(0L, 1);
        long pref = 0;
        int count = 0;

        for (int x : A) {
            pref += x;
            if (freq.containsKey(pref - K)) {
                count += freq.get(pref - K);
            }
            freq.put(pref, freq.getOrDefault(pref, 0) + 1);
        }
        return count;
    }

    // 3. Subarray Sums Divisible by K (LeetCode 974)
    public static int subarraysDivByK(int[] A, int K) {
        Map<Integer, Integer> remFreq = new HashMap<>();
        remFreq.put(0, 1);
        long pref = 0;
        int count = 0;

        for (int x : A) {
            pref += x;
            int rem = (int) ((pref % K + K) % K); // Normalized positive remainder
            if (remFreq.containsKey(rem)) {
                count += remFreq.get(rem);
            }
            remFreq.put(rem, remFreq.getOrDefault(rem, 0) + 1);
        }
        return count;
    }

    // 4. Contiguous Array - Equal 0s and 1s (LeetCode 525)
    public static int findMaxLengthEqual01(int[] nums) {
        Map<Integer, Integer> firstSeen = new HashMap<>();
        firstSeen.put(0, -1);
        int pref = 0;
        int maxLen = 0;

        for (int j = 0; j < nums.length; j++) {
            pref += (nums[j] == 1) ? 1 : -1;
            if (firstSeen.containsKey(pref)) {
                maxLen = Math.max(maxLen, j - firstSeen.get(pref));
            } else {
                firstSeen.put(pref, j);
            }
        }
        return maxLen;
    }
}
```

3 Paradigm 2: Sliding Window and Two-Pointer Search Space Pruning

When sequence elements are strictly non-negative ($a_k \geq 0$), the prefix sum sequence $P[j]$ is monotonically non-decreasing ($P[j] \geq P[j-1]$). Expanding the right boundary R increases window sum and distinct element counts, while advancing the left boundary L decreases window metrics. This monotonicity enables two-pointer search space pruning from $\mathcal{O}(N^2)$ to amortized $\mathcal{O}(N)$.

3.1 Variable Window Contraction Mechanics

3.1.1 Minimum Size Subarray Sum (LeetCode 209)

We maintain window $[L, R]$. For each R, we add $A[R]$ to $window_sum$. While $window_sum \geq Target$, we update $min_len = \min(min_len, R - L + 1)$ and contract L by subtracting $A[L]$.

Theorem 7 (Amortized Linear Complexity of Sliding Window). The two-pointer sliding window algorithm executes in $\mathcal{O}(N)$ total time complexity and $\mathcal{O}(1)$ auxiliary space.

Proof. The right pointer R increments from 0 to $N-1$ exactly N times. The left pointer L increments monotonically from 0 to $N-1$ and never decrements. Across the entire execution, L is incremented at most N times. Total pointer movements are bounded by $2N = \mathcal{O}(N)$. □

3.1.2 Subarray Product Less Than K (LeetCode 713)

For positive integer arrays, expanding R multiplies $prod$ by $A[R]$. While $prod \geq K$, we divide $prod$ by $A[L]$ and advance L.

Lemma 8 (Ending-At Count Invariant). For a valid window $[L, R]$ where all sub-windows ending at R satisfy the condition, the number of valid contiguous subarrays ending strictly at R is exactly $R - L + 1$.

Proof. The valid subarrays ending at R are $A[R \dots R], A[R-1 \dots R], \dots, A[L \dots R]$. The starting indices range from L to R, giving $(R - L + 1)$ distinct choices. □

3.2 Exact-K Inclusion-Exclusion Decomposition

3.2.1 Subarrays with K Different Integers (LeetCode 992)

Direct sliding window cannot maintain "Exactly K distinct elements" because contracting L when distinct elements equal K might miss valid subarrays starting further right.

Theorem 9 (Exact-K Decomposition Theorem). Let $f(K)$ be the number of contiguous subarrays containing at most K distinct integers. The number of subarrays containing exactly K distinct integers is:

$$\text{Exactly}(K) = f(K) - f(K-1) \tag{10}$$

Proof. Let $\mathcal{S}$ be the set of all subarrays. Let $\mathcal{D}(s)$ be the number of distinct integers in subarray s. $f(K) = |\{s \in \mathcal{S} \mid \mathcal{D}(s) \leq K\}|$. Partitioning this set:

$$\{s \mid \mathcal{D}(s) \leq K\} = \{s \mid \mathcal{D}(s) \leq K-1\} \cup \{s \mid \mathcal{D}(s) = K\} \tag{11}$$

Since these sets are disjoint:

$$f(K) = f(K-1) + \text{Exactly}(K) \implies \text{Exactly}(K) = f(K) - f(K-1) \tag{12}$$

Because $f(K)$ enforces a monotonic "at most" condition, $f(K)$ is solved in $\mathcal{O}(N)$ time using a standard sliding window. □

3.3 Sliding Window with Monotonic Double-Ended Queues

3.3.1 Longest Subarray with Absolute Difference ≤ Limit (LeetCode 1438)

When the window property depends on dynamic maximum and minimum values ($\max_{L \leq k \leq R} A[k] - \min_{L \leq k \leq R} A[k] \leq Limit$), updating min/max during L contraction requires monotonic deques:

- Max Deque: Stores elements in monotonically decreasing order. $max_dq.peekFirst()$ provides the window maximum in $\mathcal{O}(1)$.
- Min Deque: Stores elements in monotonically increasing order. $min_dq.peekFirst()$ provides the window minimum in $\mathcal{O}(1)$.

When window violation occurs ($max_dq.peekFirst() - min_dq.peekFirst() > Limit$), L advances, popping $A[L]$ from deques if matched.

3.4 Compilable Java Reference Implementations

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.Map;

public class SlidingWindowSubarraySolutions {

    // 1. Minimum Size Subarray Sum (LeetCode 209)
    public static int minSubArrayLen(int target, int[] nums) {
        int L = 0, sum = 0, minLen = Integer.MAX_VALUE;
```

```
        for (int R = 0; R < nums.length; R++) {
            sum += nums[R];
            while (sum >= target) {
                minLen = Math.min(minLen, R - L + 1);
                sum -= nums[L++];
            }
        }
        return minLen == Integer.MAX_VALUE ? 0 : minLen;
    }

    // 2. Subarrays with K Different Integers (LeetCode 992)
    public static int subarraysWithKDistinct(int[] nums, int k) {
        return atMostKDistinct(nums, k) - atMostKDistinct(nums, k - 1);
    }

    private static int atMostKDistinct(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        int L = 0, count = 0;
        for (int R = 0; R < nums.length; R++) {
            freq.put(nums[R], freq.getOrDefault(nums[R], 0) + 1);
            while (freq.size() > k) {
                freq.put(nums[L], freq.get(nums[L]) - 1);
                if (freq.get(nums[L]) == 0) freq.remove(nums[L]);
                L++;
            }
            count += (R - L + 1); // Lemma 2 ending-at count invariant
        }
        return count;
    }

    // 3. Longest Subarray with Diff <= Limit (LeetCode 1438)
    public static int longestSubarrayDiffLimit(int[] nums, int limit) {
        Deque<Integer> maxDq = new ArrayDeque<>();
        Deque<Integer> minDq = new ArrayDeque<>();
        int L = 0, maxLen = 0;

        for (int R = 0; R < nums.length; R++) {
            while (!maxDq.isEmpty() && maxDq.peekLast() < nums[R]) maxDq.pollLast();
            while (!minDq.isEmpty() && minDq.peekLast() > nums[R]) minDq.pollLast();
            maxDq.addLast(nums[R]);
            minDq.addLast(nums[R]);

            while (maxDq.peekFirst() - minDq.peekFirst() > limit) {
                if (maxDq.peekFirst() == nums[L]) maxDq.pollFirst();
                if (minDq.peekFirst() == nums[L]) minDq.pollFirst();
                L++;
            }
            maxLen = Math.max(maxLen, R - L + 1);
        }
        return maxLen;
    }
}
```

4 Paradigm 3: Dynamic Programming and Kadane's Local State Optimization

When evaluating maximum, minimum, or product optimizations over subarrays where elements may be negative, greedy sliding window invariants fail because prefix sums are non-monotonic. Dynamic programming constructs state transitions over contiguous local choices.

4.1 Classic Kadane's Algorithm (LeetCode 53)

4.1.1 State Definition and Recurrence

Let $DP[i]$ be the maximum subarray sum ending strictly at index i. Any contiguous subarray ending at index i either:

1) Consists solely of element a_i.
2) Extends the optimal contiguous subarray ending at index $i-1$ by appending a_i.

This yields the Bellman recurrence relation:

$$DP[i] = \max(a_i, DP[i-1] + a_i) \tag{13}$$

The global maximum subarray sum is $S^* = \max_{0 \le i < N} DP[i]$.

Theorem 10 (Inductive Correctness of Kadane's Recurrence). Kadane's recurrence correctly computes the maximum sum of any contiguous subarray ending at index i.

Proof. By induction on i:

- Base Case ($i = 0$): The only subarray ending at index 0 is $A[0 \dots 0]$, with sum a_0. $DP[0] = a_0$.
- Inductive Step: Assume $DP[i-1]$ is the optimal maximum sum for all contiguous subarrays ending at $i-1$. Any subarray ending at i has form $A[k \dots i]$ for $0 \le k \le i$. If $k = i$, sum is a_i. If $k < i$, sum is $S(k, i-1) + a_i$. Maximizing over $k < i$ yields $(\max_{k<i} S(k, i-1)) + a_i = DP[i-1] + a_i$. Taking $\max(a_i, DP[i-1] + a_i)$ guarantees exact optimality for $DP[i]$.

□

4.1.2 Space Optimization

Since $DP[i]$ depends only on $DP[i-1]$, auxiliary space reduces from $\mathcal{O}(N)$ array to scalar $current_max$, achieving $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.

4.2 Dual-State Propagation: Maximum Product Subarray (LeetCode 152)

Multiplying numbers by a negative scalar inverts order inequalities ($x < y \implies -2x > -2y$). A large negative product becomes a large positive product when multiplied by another negative element.

4.2.1 Dual Recurrence Equations

We maintain two states simultaneously:

- $MaxDP[i]$: Maximum product subarray ending at i.
- $MinDP[i]$: Minimum product subarray ending at i.

$$MaxDP[i] = \max\Big(a_i,\, a_i \cdot MaxDP[i-1],\, a_i \cdot MinDP[i-1]\Big) \tag{14}$$

$$MinDP[i] = \min\Big(a_i,\, a_i \cdot MaxDP[i-1],\, a_i \cdot MinDP[i-1]\Big) \tag{15}$$

4.3 Circular Topology Transformations (LeetCode 918)

For a circular array where index $N-1$ connects to 0, a maximum circular subarray takes one of two topological structures:

1) Non-Wrapping Subarray: Standard linear Kadane maximum $KadaneMax(A)$.
2) Wrapping Subarray: Extends from head and tail, leaving a contiguous middle subarray excluded. To maximize wrapping sum, we minimize the excluded middle subarray sum:

$$WrappingMax = TotalSum - KadaneMin(A) \tag{16}$$

Theorem 11 (Circular Subarray Edge Case Optimization). If all elements in A are negative, $KadaneMin(A) = TotalSum$, yielding $WrappingMax = 0$ (an empty subarray). In this scenario, the algorithm must return $KadaneMax(A)$.

4.4 Compilable Java Reference Implementations

```
public class DynamicProgrammingSubarraySolutions {

    // 1. Maximum Subarray Sum - Kadane's Algorithm (LeetCode 53)
    public static int maxSubArray(int[] nums) {
        int currentMax = nums[0];
        int globalMax = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentMax = Math.max(nums[i], currentMax + nums[i]);
            globalMax = Math.max(globalMax, currentMax);
        }
        return globalMax;
    }

    // 2. Maximum Product Subarray (LeetCode 152)
    public static int maxProduct(int[] nums) {
        int maxProd = nums[0];
        int minProd = nums[0];
        int globalMax = nums[0];

        for (int i = 1; i < nums.length; i++) {
            int x = nums[i];
            if (x < 0) { // Swap state bounds on negative factor
                int temp = maxProd;
                maxProd = minProd;
                minProd = temp;
            }
            maxProd = Math.max(x, x * maxProd);
            minProd = Math.min(x, x * minProd);
            globalMax = Math.max(globalMax, maxProd);
        }
        return globalMax;
    }

    // 3. Maximum Sum Circular Subarray (LeetCode 918)
    public static int maxSubarraySumCircular(int[] nums) {
        int totalSum = 0;
        int curMax = 0, globalMax = nums[0];
        int curMin = 0, globalMin = nums[0];

        for (int x : nums) {
            totalSum += x;
            curMax = Math.max(x, curMax + x);
            globalMax = Math.max(globalMax, curMax);

            curMin = Math.min(x, curMin + x);
            globalMin = Math.min(globalMin, curMin);
        }

        // Edge case: if all numbers are negative, globalMax is max single element
        if (globalMax < 0) return globalMax;
        return Math.max(globalMax, totalSum - globalMin);
    }
}
```

5 Paradigm 4: Monotonic Stack and Element Contribution Mechanics

Traditional brute-force algorithms iterate over all $\mathcal{O}(N^2)$ candidate subarrays and find the minimum/maximum element within each subarray. The Contribution Paradigm flips this orientation: for each element a_i, we determine the exact count of contiguous subarrays in which a_i serves as the minimum (or maximum) element.

5.1 Range Boundary Determination (PSE and NSE)

For element a_i, define its maximum extension range $[L_i, R_i]$ where a_i remains the minimum element:

- Previous Smaller Element ($PSE[i]$): The largest index $l < i$ such that $a_l < a_i$. If no such element exists, $PSE[i] = -1$.
- Next Smaller Element ($NSE[i]$): The smallest index $r > i$ such that $a_r \leq a_i$. If no such element exists, $NSE[i] = N$.

Theorem 12 (Subarray Count via Boundaries). The number of contiguous subarrays where a_i is the minimum element is:

$$Count(i) = (i - PSE[i]) \times (NSE[i] - i) \tag{17}$$

Proof. Any valid subarray $A[start \dots end]$ where a_i is the minimum must have $start \in (PSE[i], i]$ and $end \in [i, NSE[i])$. The number of left boundary choices is $i - PSE[i]$, and right boundary choices is $NSE[i] - i$. Independence of boundary selection yields the product $(i - PSE[i]) \times (NSE[i] - i)$. □

5.2 Duplicate Handling: Strict vs. Non-Strict Inequalities

If array A contains duplicate elements (e.g., $A = [2, 2]$), equal elements could be claimed as the minimum by multiple indices for the same subarray, causing overcounting.

Theorem 13 (Asymmetric Boundary Uniqueness Theorem). Defining PSE using strict inequality ($a_l < a_i$) and NSE using non-strict inequality ($a_r \leq a_i$) guarantees that every contiguous subarray has exactly ONE unique designated minimum index.

Proof. Suppose a subarray contains multiple instances of the minimum value V at indices $idx_1 < idx_2 < \cdots < idx_k$. Under non-strict NSE, idx_1 sees idx_2 as $a_{idx_2} \leq a_{idx_1}$, so $NSE[idx_1] \leq idx_2$. Thus idx_1's right extension terminates before or at idx_2. For idx_2, its PSE search moves left; under strict PSE, it does not stop at idx_1 because $a_{idx_1} = a_{idx_2}$ is not strictly smaller. However, idx_2's left bound includes idx_1. Thus, only idx_1 claims subarrays containing both idx_1 and idx_2. Duplicate counting is mathematically eliminated. □

5.3 Sum of Subarray Minimums & Ranges (LeetCode 907 & 2104)

5.3.1 Sum of Subarray Minimums (LeetCode 907)

$$\text{Total Min Sum} = \sum_{i=0}^{N-1} a_i \times (i - PSE[i]) \times (NSE[i] - i) \pmod{10^9 + 7} \tag{18}$$

5.3.2 Sum of Subarray Ranges (LeetCode 2104)

Range of subarray $A[i \dots j]$ is $\max(A[i \dots j]) - \min(A[i \dots j])$. Summing over all subarrays:

$$\sum_{\text{subarrays}} (\text{Max} - \text{Min}) = \sum_{i=0}^{N-1} a_i \cdot CountMax(i) - \sum_{i=0}^{N-1} a_i \cdot CountMin(i) \tag{19}$$

5.4 Compilable Java Reference Implementations

```
import java.util.ArrayDeque;
import java.util.Deque;

public class MonotonicStackSubarraySolutions {

    // 1. Sum of Subarray Minimums (LeetCode 907)
    public static int sumSubarrayMins(int[] arr) {
        int N = arr.length;
        int[] pse = new int[N]; // Previous Smaller Element (Strict <)
        int[] nse = new int[N]; // Next Smaller Element (Non-strict <=)

        Deque<Integer> stack = new ArrayDeque<>();

        // Compute PSE
        for (int i = 0; i < N; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) {
                stack.pop();
            }
            pse[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }
```

```
        stack.clear();

        // Compute NSE
        for (int i = N - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
                stack.pop();
            }
            nse[i] = stack.isEmpty() ? N : stack.peek();
            stack.push(i);
        }

        long totalSum = 0;
        long MOD = 1_000_000_007;

        for (int i = 0; i < N; i++) {
            long leftCount = i - pse[i];
            long rightCount = nse[i] - i;
            long contribution = (leftCount * rightCount) % MOD;
            totalSum = (totalSum + contribution * arr[i]) % MOD;
        }

        return (int) totalSum;
    }

    // 2. Sum of Subarray Ranges (LeetCode 2104)
    public static long subArrayRanges(int[] nums) {
        return getContribution(nums, true) - getContribution(nums, false);
    }

    private static long getContribution(int[] nums, boolean isMax) {
        int N = nums.length;
        int[] left = new int[N];
        int[] right = new int[N];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < N; i++) {
            while (!stack.isEmpty() && (isMax ? nums[stack.peek()] <= nums[i] : nums[stack.peek()] >= nums[i])) {
                stack.pop();
            }
            left[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }

        stack.clear();

        for (int i = N - 1; i >= 0; i--) {
            while (!stack.isEmpty() && (isMax ? nums[stack.peek()] < nums[i] : nums[stack.peek()] > nums[i])) {
                stack.pop();
            }
            right[i] = stack.isEmpty() ? N : stack.peek();
            stack.push(i);
        }

        long sum = 0;
        for (int i = 0; i < N; i++) {
            long count = (long) (i - left[i]) * (right[i] - i);
            sum += count * nums[i];
        }
        return sum;
    }
}
```

6 Paradigm 5: Divide-and-Conquer, Range Trees, and Monotonic Deques on Negative Prefixes

When subarray problem constraints break monotonicity and hash maps cannot answer range inequalities in $\mathcal{O}(1)$, advanced techniques such as Merge-Sort Divide and Conquer or Monotonic Deques on Prefix Sums are required.

6.1 Count of Range Sums (LeetCode 327)

6.1.1 Problem Formalization

Given lower and upper bounds, count subarrays satisfying $lower \le S(i,j) \le upper$. Via prefix sums:

$$lower \le P[j] - P[i-1] \le upper \iff P[j] - upper \le P[i-1] \le P[j] - lower \tag{20}$$

Because this requires counting prefix values within range $[P[j] - upper, P[j] - lower]$, standard hash maps fail.

6.1.2 Merge-Sort Divide and Conquer Algorithm

We sort the prefix sum array P recursively using merge sort. During the merge step between sorted left half $P[left \dots mid]$ and right half $P[mid+1 \dots right]$:

- For each element $P[j]$ in the right half, valid left prefix sums $P[i]$ in the left half lie in range $[P[j]-upper, P[j]-lower]$.
- Because both halves are sorted, two pointers k_1 and k_2 in the left half move monotonically right as j increases.
- Count of valid left prefixes for j is $(k_2 - k_1)$ in $\mathcal{O}(1)$ amortized time per j.

Total time complexity is $\mathcal{O}(N \log N)$ and auxiliary space is $\mathcal{O}(N)$.

6.2 Shortest Subarray with Sum at Least K (LeetCode 862)

When array A contains negative integers, prefix sum $P[j]$ is non-monotonic, preventing simple sliding windows.

6.2.1 Monotonic Deque on Prefix Sums

We maintain a deque of indices storing prefix sums $P[i]$ in strictly increasing order. For each index j:

1) Target Condition Check: While $P[j]-P[dq.peekFirst()] \ge K$, we record length $j-dq.peekFirst()$ and $dq.pollFirst()$.

   Proof of Popping First. Once $dq.peekFirst()$ satisfies the condition for index j, it will never produce a shorter valid subarray for any future index $j' > j$ (since $j' - dq.peekFirst() > j - dq.peekFirst()$). Discarding it is optimal. □

2) Obsoletion Check: While $P[j] \le P[dq.peekLast()]$, $dq.pollLast()$.

   Proof of Popping Last. $P[j]$ is smaller than $P[dq.peekLast()]$ AND located further right ($j > dq.peekLast()$). $P[j]$ is strictly superior as a starting point for future target sum subtractions and yields shorter subarray lengths. Thus $dq.peekLast()$ is dominated and can be safely purged. □

Total execution time is $\mathcal{O}(N)$ amortized.

6.3 Compilable Java Reference Implementations

```
import java.util.ArrayDeque;
import java.util.Deque;

public class AdvancedSubarraySolutions {

    // 1. Count of Range Sums (LeetCode 327) - O(N log N) Merge Sort
    public static int countRangeSum(int[] nums, int lower, int upper) {
        int N = nums.length;
        long[] pref = new long[N + 1];
        for (int i = 0; i < N; i++) pref[i + 1] = pref[i] + nums[i];
        return mergeSortAndCount(pref, 0, N + 1, lower, upper);
    }

    private static int mergeSortAndCount(long[] pref, int start, int end, int lower, int upper) {
        if (end - start <= 1) return 0;
        int mid = start + (end - start) / 2;
        int count = mergeSortAndCount(pref, start, mid, lower, upper)
                + mergeSortAndCount(pref, mid, end, lower, upper);

        int k1 = start, k2 = start;
        for (int j = mid; j < end; j++) {
```

```
            while (k1 < mid && pref[j] - pref[k1] > upper) k1++;
            while (k2 < mid && pref[j] - pref[k2] >= lower) k2++;
            count += (k2 - k1);
        }

        // Standard Merge Step
        long[] sorted = new long[end - start];
        int i = start, j = mid, idx = 0;
        while (i < mid && j < end) {
            if (pref[i] <= pref[j]) sorted[idx++] = pref[i++];
            else sorted[idx++] = pref[j++];
        }
        while (i < mid) sorted[idx++] = pref[i++];
        while (j < end) sorted[idx++] = pref[j++];
        System.arraycopy(sorted, 0, pref, start, sorted.length);

        return count;
    }

    // 2. Shortest Subarray with Sum at Least K (LeetCode 862) - O(N) Deque
    public static int shortestSubarray(int[] nums, int k) {
        int N = nums.length;
        long[] P = new long[N + 1];
        for (int i = 0; i < N; i++) P[i + 1] = P[i] + nums[i];

        int minLen = N + 1;
        Deque<Integer> dq = new ArrayDeque<>();

        for (int j = 0; j <= N; j++) {
            while (!dq.isEmpty() && P[j] - P[dq.peekFirst()] >= k) {
                minLen = Math.min(minLen, j - dq.pollFirst());
            }
            while (!dq.isEmpty() && P[j] <= P[dq.peekLast()]) {
                dq.pollLast();
            }
            dq.addLast(j);
        }
        return minLen <= N ? minLen : -1;
    }
}
```

7 Paradigm 6: Bitwise Monotonicity and Bounded State Transitions

Bitwise operations over contiguous subarrays exhibit monotonicity properties that drastically limit state complexity.

7.1 Bitwise OR Monotonicity and Bounded Set Size (LeetCode 898)

The bitwise OR operation is bit-accumulating: A OR $B \geq A$. As window length increases, bits can only transition from $0 \to 1$ and never $1 \to 0$.

Theorem 14 (30-Bit Monotonicity Bound). In a 32-bit integer sequence, as left index i moves from j down to 0, the cumulative bitwise OR value $A[i \dots j]$ can take at most 30 distinct values.

Proof. An integer has at most 30 value-bearing bits (excluding sign). Starting with a_j, extending leftwards can only set previously unset bit positions. Since set bits never clear, new distinct values occur only when at least one new bit is set to 1. There are at most 30 available bits to set, bounding distinct OR values ending at j to ≤ 30. □

7.1.1 Algorithm and Complexity

At step j, maintain a set Cur of distinct OR values of subarrays ending at $j-1$. Compute $Next = \{a_j\} \cup \{x \text{ OR } a_j \mid x \in Cur\}$. Insert $Next$ into global set Ans. Since $|Next| \leq 30$, overall time complexity is $\mathcal{O}(30N)$ and space complexity is $\mathcal{O}(30N)$.

7.2 Subarray XOR Queries (LeetCode 1310)

XOR satisfies self-inverse property ($x \oplus x = 0$). Subarray XOR is evaluated via prefix XOR array:

$$\text{SubarrayXOR}(i, j) = PrefixXOR[j] \oplus PrefixXOR[i-1] \tag{21}$$

enabling $\mathcal{O}(1)$ query response after $\mathcal{O}(N)$ preprocessing.

7.3 Compilable Java Reference Implementations

```
import java.util.HashSet;
import java.util.Set;

public class BitwiseSubarraySolutions {

    // Bitwise ORs of Subarrays (LeetCode 898) - O(30 N)
    public static int subarrayBitwiseORs(int[] arr) {
        Set<Integer> ans = new HashSet<>();
        Set<Integer> cur = new HashSet<>();

        for (int x : arr) {
            Set<Integer> next = new HashSet<>();
            next.add(x);
            for (int y : cur) {
                next.add(y | x);
            }
            ans.addAll(next);
            cur = next;
        }
        return ans.size();
    }
}
```

8 Unified Decision Framework and Master Problem Taxonomy Matrix

To provide immediate problem classification for LeetCode and competitive programming, we summarize all contiguous subarray paradigms, properties, complexities, and representative problems in Table 1.

TABLE 1
Comprehensive Contiguous Subarray Problem Taxonomy Matrix

Paradigm	Core Input Condition	Target Metric / Constraint	Time	Space	Canonical LeetCode
Prefix Hash Map	Arbitrary Integers	Sum $= K$, Longest Zero Sum	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 560, LC 525, Screenshot Zero-Sum
Prefix Remainder	Modulo K Division	Sum $(\bmod K) = 0$, Len ≥ 2	$\mathcal{O}(N)$	$\mathcal{O}(K)$	LC 974, LC 523, LC 1524
Sliding Window	Non-negative $a_i \geq 0$	Min Len Sum $\geq K$, Prod $< K$	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 209, LC 713
Exact-K Sliding	Monotonic Set Metrics	Exactly K Distinct Elements	$\mathcal{O}(N)$	$\mathcal{O}(K)$	LC 992, LC 340, LC 904
Monotonic Deque	Sliding Bounds	Max $-$ Min $\leq$ Limit	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 1438, LC 239
Kadane DP	Mixed Real Numbers	Maximum / Minimum Subarray Sum	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 53, LC 918
Dual Product DP	Negative Factors	Maximum Subarray Product	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 152
Monotonic Stack	Element Contribution	Sum of Subarray Mins / Ranges	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 907, LC 2104, LC 2334
Prefix Merge Sort	Non-monotonic Ranges	Lower $\leq$ Sum $\leq$ Upper	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	LC 327
Prefix Deque	Mixed	Shortest Subarray Sum $\geq K$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 862
Bitwise Monotonic	32-bit Integers	Distinct Bitwise OR / XOR	$\mathcal{O}(30N)$	$\mathcal{O}(30N)$	LC 898, LC 1310

8.1 Step-by-Step Problem Classification Algorithm

When presented with any new contiguous subarray problem:

1) Check Monotonicity: Are elements non-negative? If yes, apply Two-Pointer Sliding Window (§3).
2) Check Target Equality / Divisibility: Is the objective finding a subarray sum equal to K or divisible by K? Apply Prefix-Sum Hash Map (§2).
3) Check Max / Min Local Optimality: Is the objective maximizing sum/product under mixed positive/negative values? Apply Kadane Dynamic Programming (§4).
4) Check All-Subarray Aggregations: Does the metric sum minimums, maximums, or ranges across all subarrays? Apply Monotonic Stack Contribution Paradigm (§5).
5) Check Negative Prefixes + Target Bounds: Does the problem ask for range sums or shortest target sum with negative elements? Apply Merge Sort Divide and Conquer or Monotonic Deque on Prefixes (§6).
6) Check Bitwise Metrics: Does the problem involve bitwise OR or AND? Apply 30-Bit Monotonic Accumulation (§7).

By adhering to this unified mathematical framework and algorithmic classification, all contiguous subarray problems across LeetCode and advanced competitive programming can be systematically solved with optimal asymptotic time and space bounds.